Chapter 10. Deployments

So far, you have seen how to package your applications as containers, create replicated sets of containers, and use Ingress controllers to load balance traffic to your services. You can use all of these objects (Pods, ReplicaSets, and Services) to build a single instance of your application. However, they do little to help you manage the daily or weekly cadence of releasing new versions of your application. Indeed, both Pods and ReplicaSets are expected to be tied to specific container images that don't change.

The Deployment object exists to manage the release of new versions. Deployments represent deployed applications in a way that transcends any particular version. Additionally, Deployments enable you to easily move from one version of your code to the next. This "rollout" process is specifiable and careful. It waits for a user-configurable amount of time between upgrading individual Pods. It also uses health checks to ensure that the new version of the application is operating correctly and stops the deployment if too many failures occur.

Using Deployments, you can simply and reliably roll out new software versions without downtime or errors. The actual mechanics of the software rollout performed by a Deployment are controlled by a Deployment controller that runs in the Kubernetes cluster itself. This means you can let a Deployment proceed unattended and it will still operate correctly and safely. This makes it easy to integrate Deployments with numerous continuous delivery tools and services. Further, running server-side makes it safe to perform a rollout from places with poor or intermittent internet connectivity. Imagine rolling out a new version of your software from your phone while riding on the subway. Deployments make this possible and safe!

NOTE

When Kubernetes was first released, one of the most popular demonstrations of its power was the "rolling update," which showed how you could use a single command to seamlessly update a running application without any downtime and without losing requests. This original demo was based on the `kubectl rolling-update` command, which is still available in the command-line tool, although its functionality has largely been subsumed by the Deployment object.

Your First Deployment

Like all objects in Kubernetes, a Deployment can be represented as a declarative YAML object that provides the details about what you want to run. In the following case, the Deployment is requesting a single instance of the `kuard` application:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kuard
  labels:
    run: kuard
spec:
  selector:
    matchLabels:
      run: kuard
  replicas: 1
  template:
    metadata:
      labels:
        run: kuard
    spec:
      containers:
      - name: kuard
        image: gcr.io/kuar-demo/kuard-amd64:blue
```

Save this YAML file as *kuard-deployment.yaml*, then you can create it using:

```
$ kubectl create -f kuard-deployment.yaml
```

Let's explore how Deployments actually work. Just as we learned that ReplicaSets manage Pods, Deployments manage ReplicaSets. As with all relationships in Kubernetes, this relationship is defined by labels and a label selector. You can see the label selector by looking at the Deployment object:

```
$ kubectl get deployments kuard \
  -o jsonpath --template {.spec.selector.matchLabels}

{"run":"kuard"}
```

From this you can see that the Deployment is managing a ReplicaSet with the label `run=kuard`. You can use this in a label selector query across ReplicaSets to find that specific ReplicaSet:

```
$ kubectl get replicasets --selector=run=kuard

NAME          DESIRED  CURRENT  READY   AGE
kuard-1128242161 1       1       1       13m
```

Now let's look at the relationship between a Deployment and a ReplicaSet in action. We can resize the Deployment using the imperative `scale` command:

```
$ kubectl scale deployments kuard --replicas=2

deployment.apps/kuard scaled
```

Now if we list that ReplicaSet again, we should see:

```
$ kubectl get replicasets --selector=run=kuard

NAME          DESIRED  CURRENT  READY   AGE
kuard-1128242161 2       2       2       13m
```

Scaling the Deployment has also scaled the ReplicaSet it controls.

Now let's try the opposite, scaling the ReplicaSet:

```
$ kubectl scale replicasets kuard-1128242161 --replicas=1

replicaset.apps/kuard-1128242161 scaled
```

Now `get` that ReplicaSet again:

```
$ kubectl get replicasets --selector=run=kuard

NAME          DESIRED  CURRENT  READY   AGE
kuard-1128242161 2       2       2       13m
```

That's odd. Despite scaling the ReplicaSet to one replica, it still has two replicas as its desired state. What's going on?

Remember, Kubernetes is an online, self-healing system. The top-level Deployment object is managing this ReplicaSet. When you adjust the number of replicas to one, it no longer matches the desired state of the Deployment, which has `replicas` set to `2`. The Deployment controller notices this and takes action to ensure the observed state matches the desired state, in this case readjusting the number of replicas back to two.

If you ever want to manage that ReplicaSet directly, you need to delete the Deployment. (Remember to set `--cascade` to `false`, or else it will delete the ReplicaSet and Pods as well!)

Creating Deployments

Of course, as stated in the introduction, you should have a preference for declarative management of your Kubernetes configurations. This means maintaining the state of your Deployments in YAML or JSON files on disk.

As a starting point, download this Deployment into a YAML file:

```
$ kubectl get deployments kuard -o yaml > kuard-deployment.yaml
$ kubectl replace -f kuard-deployment.yaml --save-config
```

If you look in the file, you will see something like this (note that we've removed a lot of read-only and default fields for readability). Pay attention to the annotations, selector, and strategy fields as they provide insight into Deployment-specific functionality:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: null
  generation: 1
  labels:
    run: kuard
  name: kuard
spec:
  progressDeadlineSeconds: 600
```

```
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      run: kuard
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
      labels:
        run: kuard
    spec:
      containers:
      - image: gcr.io/kuar-demo/kuard-amd64:blue
        imagePullPolicy: IfNotPresent
        name: kuard
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status: {}
```

NOTE

You also need to run `kubectl replace --save-config`. This adds an annotation so that, when applying changes in the future, `kubectl` will know what the last applied configuration was for smarter merging of configs. If you always use `kubectl apply`, this step is only required after the first time you create a Deployment using `kubectl create -f`.

The Deployment spec has a very similar structure to the ReplicaSet spec. There is a Pod template, which contains a number of containers that are created for each replica managed by the Deployment. In addition to the Pod specification, there is also a `strategy` object:

```
...
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
...
```

The `strategy` object dictates the different ways in which a rollout of new software can proceed. There are two strategies supported by Deployments: `Recreate` and `RollingUpdate`. These are discussed in detail later in this chapter.

Managing Deployments

As with all Kubernetes objects, you can get detailed information about your Deployment via the `kubectl describe` command. This command provides an overview of the Deployment configuration, which includes interesting fields like the Selector, Replicas, and Events:

```
$ kubectl describe deployments kuard

Name:                   kuard
Namespace:              default
CreationTimestamp:      Tue, 01 Jun 2021 21:19:46 -0700
Labels:                 run=kuard
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               run=kuard
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 ...
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  run=kuard
  Containers:
   kuard:
    Image:        gcr.io/kuar-demo/kuard-amd64:blue
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
```

```
  Type        Status  Reason
  ----        ------  ------
  Available   True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   kuard-6d69d9fc5c (2/2 replicas created)
Events:
  Type    Reason             Age                From                    Message
  ----    ------             ----               ----                    -------
  Normal  ScalingReplicaSet  4m6s               deployment-con...  ...
  Normal  ScalingReplicaSet  113s (x2 over 3m20s)  deployment-con...  ...
```

In the output of `describe`, there is a great deal of important information. Two of the most important pieces of information in the output are `OldReplicaSets` and `NewReplicaSet`. These fields point to the ReplicaSet objects this Deployment is currently managing. If a Deployment is in the middle of a rollout, both fields will be set to a value. If a rollout is complete, `OldReplicaSets` will be set to `<none>`.

In addition to the `describe` command, there is also the `kubectl rollout` command for Deployments. We will go into this command in more detail later on, but for now, know that you can use `kubectl rollout history` to obtain the history of rollouts associated with a particular Deployment. If you have a current Deployment in progress, you can use `kubectl rollout status` to obtain the current status of that rollout.

Updating Deployments

Deployments are declarative objects that describe a deployed application. The two most common operations on a Deployment are scaling and application updates.

Scaling a Deployment

Although we previously showed how to imperatively scale a Deployment using the `kubectl scale` command, the best practice is to manage your Deployments declaratively via the YAML files, then use those files to update your Deployment. To scale up a Deployment, you would edit your YAML file to increase the number of replicas:

```
...
spec:
  replicas: 3
...
```

Once you have saved and committed this change, you can update the Deployment using the `kubectl apply` command:

```
$ kubectl apply -f kuard-deployment.yaml
```

This will update the desired state of the Deployment, causing it to increase the size of the ReplicaSet it manages and eventually create a new Pod managed by the Deployment:

```
$ kubectl get deployments kuard

NAME   READY  UP-TO-DATE  AVAILABLE  AGE
kuard  3/3    3           3          10m
```

Updating a Container Image

The other common use case for updating a Deployment is to roll out a new version of the software running in one or more containers. To do this, you should likewise edit the Deployment YAML file, though in this case you are updating the container image, rather than the number of replicas:

```
...
    containers:
    - image: gcr.io/kuar-demo/kuard-amd64:green
      imagePullPolicy: Always
...
```

Annotate the template for the Deployment to record some information about the update:

```
...
spec:
  ...
  template:
    metadata:
      annotations:
        kubernetes.io/change-cause: "Update to green kuard"
...
```

CAUTION

Make sure you add this annotation to the template and not the Deployment itself, since the `kubectl apply` command uses this field in the Deployment object. Also, do not update the `change-cause` annotation when doing simple scaling operations. A modification of `change-cause` is a significant change to the template and will trigger a new rollout.

Again, you can use `kubectl apply` to update the Deployment:

```
$ kubectl apply -f kuard-deployment.yaml
```

After you update the Deployment, it will trigger a rollout, which you can then monitor via the `kubectl rollout` command:

```
$ kubectl rollout status deployments kuard
deployment "kuard" successfully rolled out
```

You can see the old and new ReplicaSets managed by the Deployment along with the images being used. Both the old and new ReplicaSets are kept around in case you want to roll back:

```
$ kubectl get replicasets -o wide

NAME           DESIRED  CURRENT  READY  ...  IMAGE(S)          ...
kuard-1128242161  0        0        0      ...  gcr.io/kuar-demo/  ...
kuard-1128635377  3        3        3      ...  gcr.io/kuar-demo/  ...
```

If you are in the middle of a rollout and you want to temporarily pause it (e.g., if you start seeing weird behavior in your system that you want to investigate), you can use the `pause` command:

```
$ kubectl rollout pause deployments kuard
deployment.apps/kuard paused
```

If, after investigation, you believe the rollout can safely proceed, you can use the `resume` command to start up where you left off:

```
$ kubectl rollout resume deployments kuard
deployment.apps/kuard resumed
```

Rollout History

Kubernetes Deployments maintain a history of rollouts, which can be useful both for understanding the previous state of the Deployment and for rolling back to a specific version.

You can see the Deployment history by running:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION  CHANGE-CAUSE
1         <none>
2         Update to green kuard
```

The revision history is given in oldest to newest order. A unique revision number is incremented for each new rollout. So far we have two: the initial Deployment and the update of the image to `kuard:green`.

If you are interested in more details about a particular revision, you can add the `--revision` flag to view details about that specific revision:

```
$ kubectl rollout history deployment kuard --revision=2

deployment.apps/kuard with revision #2
Pod Template:
  Labels:       pod-template-hash=54b74ddcd4
        run=kuard
  Annotations:  kubernetes.io/change-cause: Update to green kuard
  Containers:
   kuard:
    Image:      gcr.io/kuar-demo/kuard-amd64:green
    Port:       <none>
    Host Port:  <none>
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
```

Let's do one more update for this example. Update the `kuard` version back to `blue` by modifying the container version number and updating the `change-cause` annotation. Apply it with `kubectl apply`. The history should now have three entries:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION  CHANGE-CAUSE
1         <none>
2         Update to green kuard
3         Update to blue kuard
```

Let's say there is an issue with the latest release and you want to roll back while you investigate. You can simply undo the last rollout:

```
$ kubectl rollout undo deployments kuard
deployment.apps/kuard rolled back
```

The `undo` command works regardless of the stage of the rollout. You can undo both partially completed and fully completed rollouts. An undo of a rollout is actually simply a rollout in reverse (for example from v2 to v1, instead of from v1 to v2), and all of the same policies that control the rollout strategy apply to the undo strategy as well. You can see that the Deployment object simply adjusts the desired replica counts in the managed ReplicaSets:

```
$ kubectl get replicasets -o wide

NAME           DESIRED   CURRENT   READY   ...   IMAGE(S)          ...
kuard-1128242161   0         0         0       ...   gcr.io/kuar-demo/   ...
kuard-1570155864   0         0         0       ...   gcr.io/kuar-demo/   ...
kuard-2738859366   3         3         3       ...   gcr.io/kuar-demo/   ...
```

CAUTION

When using declarative files to control your production systems, you should, as much as possible, ensure that the checked-in manifests match what is actually running in your cluster. When you do a `kubectl rollout undo`, you are updating the production state in a way that isn't reflected in your source control.

An alternative (and perhaps preferable) way to undo a rollout is to revert your YAML file and `kubectl apply` the previous version. In this way, your "change tracked configuration" more closely tracks what is really running in your cluster.

Let's look at the Deployment history again:

```
$ kubectl rollout history deployment kuard

deployment.apps/kuard
REVISION  CHANGE-CAUSE
1         <none>
3         Update to blue kuard
4         Update to green kuard
```

Revision 2 is missing! It turns out that when you roll back to a previous revision, the Deployment simply reuses the template and renumbers it so that it is the latest revision. What was revision 2 before is now revision 4.

We previously saw that you can use the `kubectl rollout undo` command to roll back to a previous version of a Deployment. Additionally, you can roll back to a specific revision in the history using the `--to-revision` flag:

```
$ kubectl rollout undo deployments kuard --to-revision=3
deployment.apps/kuard rolled back
$ kubectl rollout history deployment kuard
deployment.apps/kuard
REVISION  CHANGE-CAUSE
1         <none>
4         Update to green kuard
5         Update to blue kuard
```

Again, the `undo` took revision 3, applied it, and renumbered it as revision 5.

Specifying a revision of `0` is a shorthand way of specifying the previous revision. In this way, `kubectl rollout undo` is equivalent to `kubectl rollout undo --to-revision=0`.

By default, the last 10 revisions of a Deployment are kept attached to the Deployment object itself. It is recommended that if you have Deployments that you expect to keep around for a long time, you set a maximum history size for the Deployment revision history. For example, if you do a daily update, you may limit your revision history to 14, to keep a maximum of two weeks' worth of revisions (if you don't expect to need to roll back beyond two weeks).

To accomplish this, use the `revisionHistoryLimit` property in the Deployment specification:

```
...
spec:
  # We do daily rollouts, limit the revision history to two weeks of
  # releases as we don't expect to roll back beyond that.
  revisionHistoryLimit: 14
...
```

Deployment Strategies

When it comes time to change the version of the software implementing your service, a Kubernetes deployment supports two different rollout strategies, `Recreate` and `RollingUpdate`. Let's look at each in turn.

Recreate Strategy

The `Recreate` strategy is the simpler of the two. It simply updates the ReplicaSet it manages to use the new image and terminates all of the Pods associated with the Deployment. The ReplicaSet notices that it no longer has any replicas and re-creates all Pods using the new image. Once the Pods are re-created, they are running the new version.

While this strategy is fast and simple, it will result in workload downtime. Because of this, the `Recreate` strategy should be used only for test Deployments where a service downtime is acceptable.

RollingUpdate Strategy

The `RollingUpdate` strategy is the generally preferable strategy for any user-facing service. While it is slower than `Recreate`, it is also significantly more sophisticated and robust. Using `RollingUpdate`, you can roll out a new version of your service while it is still receiving user traffic, without any downtime.

As you might infer from the name, the `RollingUpdate` strategy works by updating a few Pods at a time, moving incrementally until all of the Pods are running the new version of your software.

Managing multiple versions of your service

Importantly, this means that for a while, both the new and the old version of your service will be receiving requests and serving traffic. This has important implications for how you build your software. Namely, it is critically important that each version of your software, and each of its clients, is capable of talking interchangeably with both a slightly older and a slightly newer version of your software.

Consider the following scenario: you are in the middle of rolling out your frontend software; half of your servers are running version 1, and half are running version 2. A user makes an initial request to your service and downloads a client-side JavaScript library that implements your UI. This request is serviced by a version 1 server, and thus the user receives the version 1 client library. This client library runs in the user's browser and makes subsequent API requests to your service. These API requests happen to be routed to a version 2 server; thus, version 1 of your JavaScript client library is talking to version 2 of your API server. If you haven't ensured compatibility between these versions, your application won't function correctly.

At first, this might seem like an extra burden. But in truth, you always had this problem; you may just not have noticed. Concretely, a user can make a request at time `t` just before you initiate an update. This request is serviced by a version 1 server. At `t_1`, you update your service to version 2. At `t_2`, the version 1 client code running on the user's browser runs and hits an API endpoint being operated by a version 2 server. No matter how you update your software, you have to maintain backward and forward compatibility for reliable updates. The nature of the `RollingUpdate` strategy simply makes that more clear and explicit.

This doesn't just apply to JavaScript clients—it's true of client libraries that are compiled into other services that make calls to your service. Just because you updated doesn't

mean they have updated their client libraries. This sort of backward compatibility is critical to decoupling your service from systems that depend on your service. If you don't formalize your APIs and decouple yourself, you are forced to carefully manage your rollouts with all of the other systems that call into your service. This kind of tight coupling makes it extremely hard to produce the necessary agility to be able to push out new software every week, let alone every hour or every day. In the decoupled architecture shown in Figure 10-1, the frontend is isolated from the backend via an API contract and a load balancer, whereas in the coupled architecture, a thick client compiled into the frontend is used to connect directly to the backends.

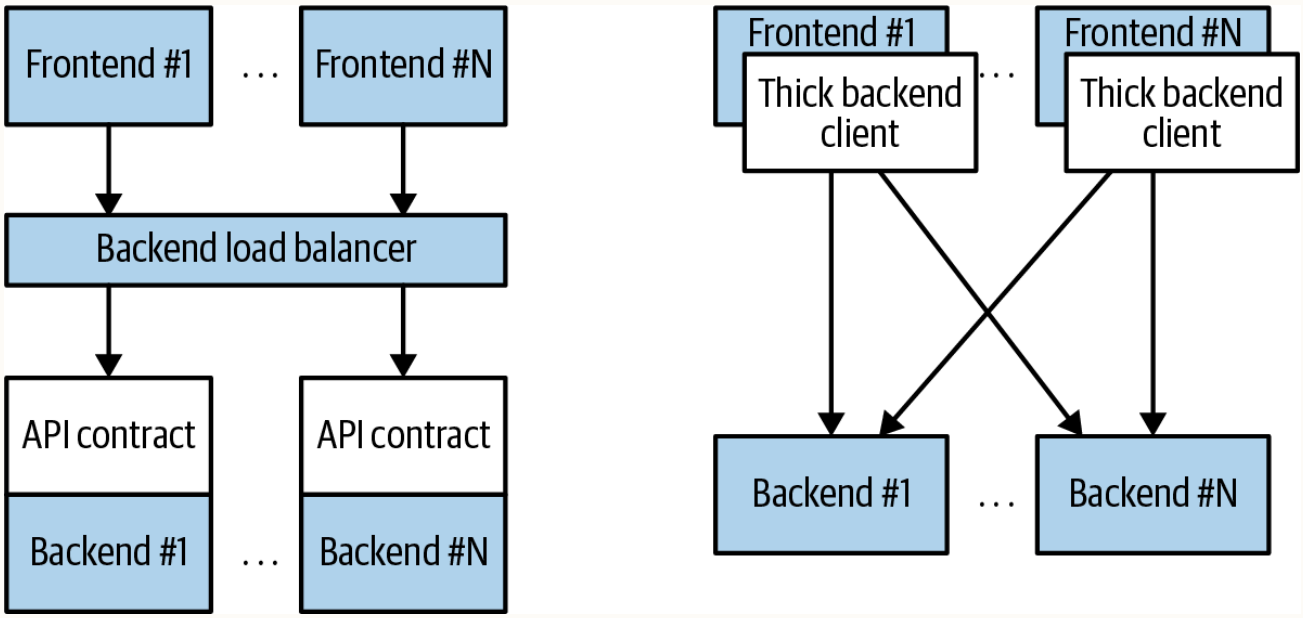


Figure 10-1. Diagrams of decoupled (left) and coupled (right) application architectures

Configuring a rolling update

`RollingUpdate` is a fairly generic strategy; it can be used to update a variety of applications in a variety of settings. Consequently, the rolling update itself is quite configurable; you can tune its behavior to suit your particular needs. There are two parameters you can use to tune the rolling update behavior: `maxUnavailable` and `maxSurge`.

The `maxUnavailable` parameter sets the maximum number of Pods that can be unavailable during a rolling update. It can either be set to an absolute number (e.g., `3`, meaning a maximum of three Pods can be unavailable) or to a percentage (e.g., `20%`, meaning a maximum of 20% of the desired number of replicas can be unavailable). Generally speaking, using a percentage is a good approach for most services, since the value is correctly applied regardless of the desired number of replicas in the Deployment. However, there are times when you may want to use an absolute number (e.g., limiting the maximum unavailable Pods to one).

At its core, the `maxUnavailable` parameter helps tune how quickly a rolling update proceeds. For example, if you set `maxUnavailable` to `50%`, then the rolling update will immediately scale the old ReplicaSet down to 50% of its original size. If you have four replicas, it will scale it down to two replicas. The rolling update will then replace the removed Pods by scaling the new ReplicaSet up to two replicas, for a total of four replicas (two old, two new). It will then scale the old ReplicaSet down to zero replicas, for a total size of two new replicas. Finally, it will scale the new ReplicaSet up to four replicas, completing the rollout. Thus, with `maxUnavailable` set to `50%`, the rollout completes in four steps, but with only 50% of the service capacity at times.

Consider what happens if we instead set `maxUnavailable` to `25%`. In this situation, each step is only performed with a single replica at a time and thus it takes twice as many steps for the rollout to complete, but availability only drops to a minimum of 75% during the rollout. This illustrates how `maxUnavailable` allows us to trade rollout speed for availability.

NOTE

The observant among you will notice that the `Recreate` strategy is identical to the `RollingUpdate` strategy with `maxUnavailable` set to `100%`.

Using reduced capacity to achieve a successful rollout is useful either when your service has cyclical traffic patterns (for example, if there's much less traffic at night) or when you have limited resources, so scaling to larger than the current maximum number of replicas isn't possible.

However, there are situations where you don't want to fall below 100% capacity, but you are willing to temporarily use additional resources to perform a rollout. In these situations, you can set the `maxUnavailable` parameter to `0`, and instead control the rollout using the `maxSurge` parameter. Like `maxUnavailable`, `maxSurge` can be specified either as a specific number or a percentage.

The `maxSurge` parameter controls how many extra resources can be created to achieve a rollout. To illustrate how this works, imagine a service with 10 replicas. We set `maxUnavailable` to `0` and `maxSurge` to `20%`. The first thing the rollout will do is scale the new ReplicaSet up by 2 replicas, for a total of 12 (120%) in the service. It will then scale the old ReplicaSet down to 8 replicas, for a total of 10 (8 old, 2 new) in the service. This process proceeds until the rollout is complete. At any time, the capacity of

the service is guaranteed to be at least 100% and the maximum extra resources used for the rollout are limited to an additional 20% of all resources.

NOTE

Setting `maxSurge` to `100%` is equivalent to a blue/green Deployment. The Deployment controller first scales the new version up to 100% of the old version. Once the new version is healthy, it immediately scales the old version down to 0%.

Slowing Rollouts to Ensure Service Health

Staged rollouts are meant to ensure that the rollout results in a healthy, stable service running the new software version. To do this, the Deployment controller always waits until a Pod reports that it is ready before moving on to update the next Pod.

WARNING

The Deployment controller examines the Pod's status as determined by its readiness checks. Readiness checks are part of the Pod's health checks, described in detail in Chapter 5. If you want to use Deployments to reliably roll out your software, you *have* to specify readiness health checks for the containers in your Pod. Without these checks, the Deployment controller is running without knowing the Pod's status.

Sometimes, however, simply noticing that a Pod has become ready doesn't give you sufficient confidence that the Pod is actually behaving correctly. Some error conditions don't occur immediately. For example, you could have a serious memory leak that takes a few minutes to show up, or you could have a bug that is only triggered by 1% of all requests. In most real-world scenarios, you want to wait a period of time to have high confidence that the new version is operating correctly before you move on to updating the next Pod.

For Deployments, this time to wait is defined by the `minReadySeconds` parameter:

```
...
spec:
  minReadySeconds: 60
...
```

Setting `minReadySeconds` to `60` indicates that the Deployment must wait for 60 seconds *after* seeing a Pod become healthy before moving on to updating the next Pod.

In addition to waiting for a Pod to become healthy, you also want to set a timeout that limits how long the system will wait. Suppose, for example, the new version of your service has a bug and immediately deadlocks. It will never become ready, and in the absence of a timeout, the Deployment controller will stall your rollout forever.

The correct behavior in such a situation is to time out the rollout. This in turn marks the rollout as failed. This failure status can be used to trigger alerting that can indicate to an operator that there is a problem with the rollout.

NOTE

At first blush, timing out a rollout might seem like an unnecessary complication. However, increasingly, things like rollouts are being triggered by fully automated systems with little to no human involvement. In such a situation, timing out becomes a critical exception, which can either trigger an automated rollback of the release or create a ticket/event that triggers human intervention.

In order to set the timeout period, you will use the Deployment parameter `progressDeadlineSeconds`:

```
...
spec:
  progressDeadlineSeconds: 600
...
```

This example sets the progress deadline to 10 minutes. If any particular stage in the rollout fails to progress in 10 minutes, then the Deployment is marked as failed, and all attempts to move the Deployment forward are halted.

It is important to note that this timeout is given in terms of Deployment *progress*, not the overall length of a Deployment. In this context, progress is defined as any time the Deployment creates or deletes a Pod. When that happens, the timeout clock is reset to zero. Figure 10-2 shows the Deployment life cycle.

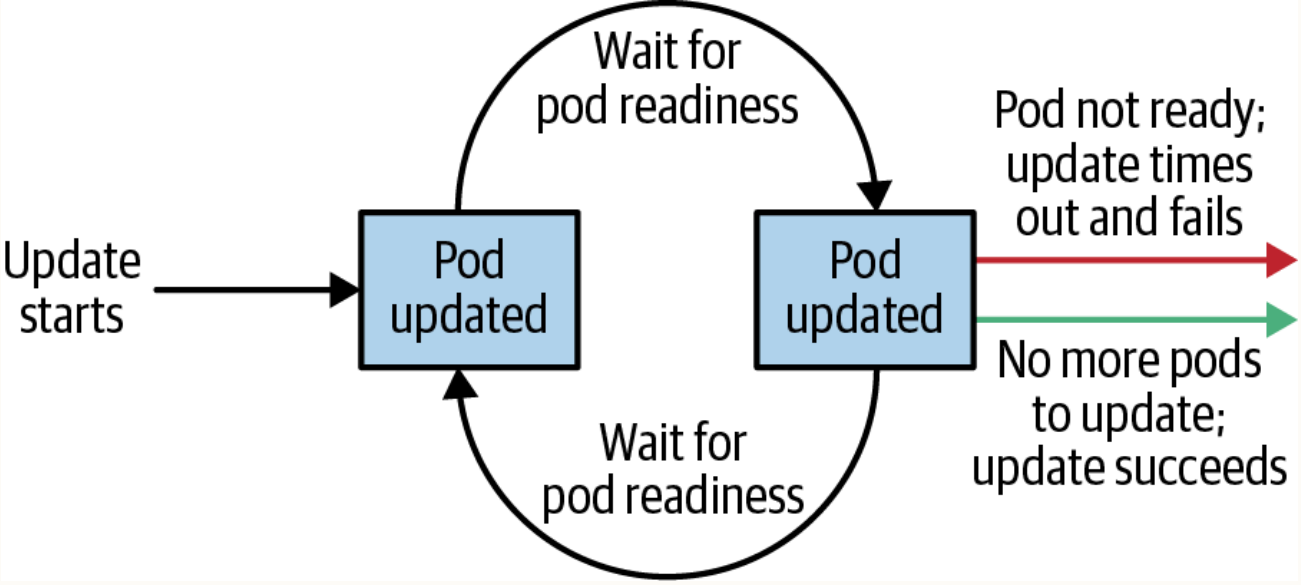


Figure 10-2. The Kubernetes Deployment life cycle

Deleting a Deployment

If you ever want to delete a Deployment, you can do it with the imperative command:

```
$ kubectl delete deployments kuard
```

You can also do it using the declarative YAML file you created earlier:

```
$ kubectl delete -f kuard-deployment.yaml
```

In either case, by default, deleting a Deployment deletes the entire service. The means it will delete not just the Deployment, but also any ReplicaSets it manages, as well as any Pods the ReplicaSets manage. As with ReplicaSets, if this is not the desired behavior, you can use the `--cascade=false` flag to delete only the Deployment object.

Monitoring a Deployment

If a Deployment fails to make progress after a specified amount of time, it will time out. When this happens, the status of the Deployment will transition to a failed state. This status can be obtained from the `status.conditions` array, where there will be a `Condition` whose `Type` is `Progressing` and whose `Status` is `False`. A Deployment in such a state has failed and will not progress further. To set how long the Deployment controller should wait before transitioning into this state, use the `spec.progressDeadlineSeconds` field.

Summary

Ultimately, the primary goal of Kubernetes is to make it easy for you to build and deploy reliable distributed systems. This means not just instantiating the application once, but managing the regularly scheduled rollout of new versions of that software service. Deployments are a critical piece of reliable rollouts and rollout management for your services. In the next chapter we will cover DaemonSets, which ensure only a single copy of a Pod is running across a set of nodes in a Kubernetes cluster.

Previous chapter
< 9. ReplicaSets

Next chapter
11. DaemonSets >